

ENMGT 5400 - Project 4: Rock, Paper, Scissors

Kelly Yue

March 23, 2026

Overview

The task for this assignment was to use the ESP32-S3 Sense micro-controller to capture and stream images from the laptop to detect rock, paper, and scissors position. How this was implemented is that a CNN is trained on the laptop using images which was captured by the ESP32-S3 camera. It is then streamed to the laptop for real-time inference. The workflow is as follows: the ESP32 captures the picture of rock, paper, or scissors which is categorized by said folders, the data is preprocessed, the model is ran, and it prints the results over USB serial to the laptop.

Pre-processing Pipeline

To pre-process the data, the model is trained on the laptop but runs on the ESP32. As a result, it was important that both sides pre-process images in the same way. The flow is as follows: the ESP32 camera takes a raw 96 x 96 gray scale BMP image, the image is resized down to 32 x 32 using nearest-neighbor down-sampling, a binary threshold is applied at 128 (this means that pixels at or above 128 becomes white which is suppose to represent the hand, otherwise pixels below becomes black which is suppose to represent the background), the 1078-byte BMP file header is stripped, leaving 1023 raw pixel bytes which is what the model expects as input. Note that matching the retraining and inference pipelines is important as mismatch can cause accuracy to go down.

CNN Architecture

When creating the model, it uses three convolutional layers, which is then followed by dense layers. This is due to TinyMaix inference engine on the ESP32 does not support MaxPooling layers, strided convolutions with stride=2 are used instead to get the same spatial down-sampling effect.

Layer	Output Shape	Purpose
Conv2D(16, 3×3, stride=2)	16×16×16	Detect basic edges and shapes
Conv2D(32, 3×3, stride=2)	8×8×32	Learn feature combinations
Conv2D(64, 3×3, stride=2)	4×4×64	Learn high-level hand patterns
Flatten	1024	Convert feature maps to 1D
Dropout(0.4)	1024	Prevent overfitting
Dense(32, ReLU)	32	Combine all features
Dense(3, Softmax)	3	Output class probabilities

Note that each convolutional layer halves the spatial dimensions. The final output is three probabilities that sun to 1.0, one for each class: rock, paper, scissors. The model was trained for up to 50

epochs using the data augmentation and class weights to deal with the imbalances in the dataset. It got a final validation accuracy of 90.1%. In terms of deciding how to craft the CNN, there is a dropout rate of 0.4 which attempts to prevent overfitting on the small dataset. The dataset is 351 images for paper, 351 images for rock, and 203 images for scissors. This imbalance in dataset was dealt with again data augmentation and class weights.

Technical Challenges

One challenge which I faced was the requirement for Wi-Fi streaming as a communication method. Although the ESP32-S3 successfully connected to the network, streaming image data proved was not possible with my Apple ecosystem due to Access Point (AP) Isolation. This security feature on iPhone hotspots prevents local peer-to-peer communication, meaning the MacBook and the micro-controller could not "see" each other or share data despite being on the same subnet. To overcome this but still fulfill the real-time requirements of the rubric, I moved to a High-Speed Binary Serial Stream. By implementing a custom synchronization protocol using Magic Bytes (0xAA 0xBB) and a 921,600 baud rate, I successfully fulfilled the requirement for a real-time data stream.

Another challenge involved the integration of the `emlearn_cnn_fp32` library with the custom MicroPython firmware. The system had multiple `ImportError` and `RuntimeError` crashes due to Application Binary Interface version mismatch between the compiled `.mpy` library and the ESP32-S3 MicroPython interpreter. Specifically, the firmware required 0x2b06 ABI (Version 6), while standard builds defaulted to incompatible versions. There was also version discrepancy between the TinyMaix conversion scripts on the laptop and the inference engine on the micro controller resulted in incompatible `.tmdl` file headers. This was resolved by matching the tool chain and the MAIX header format and having all input data formatted as an `array.array('B')` to meet the library's low-level memory requirements.

Improvements

There are many things which can be done to improve this rock, paper, scissors detector. The dataset had around 500 images total across all three classes. However having a consistent number of can help balance the dataset, and prevent overfitting for some labels (rock and paper). Additionally, there can be better down sampling. Currently, the pipeline uses nearest-neighbor which takes every 3rd pixel. Averaging the pixels in each 3 x 3 block before down sampling would preserve more structural information, this also help improve accuracy, especially for fine features such as finger gaps in scissors. Another thing which can be improved is having adaptive thresholding. This is because a fixed threshold of 128 struggles with uneven lighting. By providing adaptive thresholding, it helps adjust the cutoff locally based on the neighboring pixels. This makes the results more robust to lighting variations and different shadows. Next time I would have a None or Background class which would help the camera recognize and train on seeing a blank wall. Because the model current has to choose one of the three gestures, there may be false positives when there is not a gesture facing the camera. Lastly, when converting the model from fp32 to int8 reduces the model size by 4 x 4. This would allow a larger model to fit within the esp32's memory, which is the current main constrain on the models complexity.

Citations

- Professor provided 'Wifi.py', 'camera_test.py', 'image_preprocessing.py', the MicroPython firmware, and the 'emlearn_cnn_fp32' module
- Claude AI (Anthropic) assisted with CNN architecture design, training pipeline, emlearn model conversion, and serial viewer script
- All code was understood, integrated, and tested by the student